SPE Major Project Report

Bhavya Kapadia (IMT2022095) and Siddeshwar Kagatikar (IMT2022026)

December 2025

1 Smart Emergency Response System

Smart Emergency Response System is an MLOps-based, microservices-driven emergency management platform designed to simulate real-world emergency response operations. The system is built using **Spring Boot**, **Docker**, **Kubernetes**, and **Horizontal Pod Autoscaling (HPA)**, and integrates a full **MLOps pipeline using DVC** to dynamically generate ambulance pricing using machine learning models.

1.1 GitHub Link

`https://github.com/BKAPADIA04/SPE_Major_Project/tree/mlflow`

1.2 DockerHub Link

`https://hub.docker.com/repository/docker/bkapadia04/emergency-service/general`

`https://hub.docker.com/repository/docker/bkapadia04/dispatch-service`

`https://hub.docker.com/repository/docker/bkapadia04/ambulance-service/general`

1.3 Project Overview

The project demonstrates a cloud-native architecture where multiple independently deployable microservices collaborate to handle emergency requests efficiently and at scale. The platform supports automated scaling, centralized logging, and versioned machine learning pipelines to ensure reliability, observability, and reproducibility.

1.4 System Components

- **Ambulance Service** – Calculates ambulance pricing using a deployed machine learning model.
- **Dispatch Service** – Assigns available ambulances and communicates with the Ambulance Service.

- **Emergency Service** – Acts as the entry point for all emergency requests from users.
- **ELK Stack (Elasticsearch, Logstash, Kibana)** – Provides centralized logging and monitoring across all microservices.
- **Kubernetes** – Manages container orchestration, deployment, and service discovery.
- **Horizontal Pod Autoscaler (HPA)** – Automatically scales services based on CPU utilization.
- **MLOps Pipeline with DVC** – Enables version control of datasets, machine learning models, and training pipelines for ambulance price prediction.

1.5 System Architecture

The Smart Emergency Response System follows a microservices-based, cloud-native architecture integrated with a complete end-to-end MLOps workflow.

1.5.1 High-Level Architecture

- Users initiate emergency requests via the **Emergency Service**.
- Requests are routed internally between microservices using REST-based communication.
- The **Dispatch Service** coordinates ambulance assignment.
- The **Ambulance Service** generates real-time pricing predictions using a machine learning model.
- All services run as containerized workloads inside a **Kubernetes cluster**.
- **HPA** dynamically scales pods based on system load.
- Logs from all services are aggregated and visualized using the **ELK stack**.

2 Why Building This Project is Challenging

Developing the **Smart Emergency Response System (SPE)** presents several technical and architectural challenges due to its distributed, cloud-native design and the dynamic nature of emergency response workloads.

2.1 Dynamic Scaling of Execution Services

- Emergency requests trigger multiple backend operations such as dispatch coordination and machine learning–based pricing, leading to highly variable and unpredictable CPU and memory consumption.
- Designing an effective **Horizontal Pod Autoscaler (HPA)** is challenging because scaling decisions must respond to real-time load while preventing both over-provisioning and under-scaling.
- Excessive scaling wastes cluster resources, while insufficient scaling can delay emergency handling, making it critical to balance system responsiveness and resource efficiency.

2.2 Coordinating Multiple Microservices at Scale

- The system consists of multiple interacting microservices, including the **Emergency Service**, **Dispatch Service**, **Ambulance Service**, and observability components.
- As the system scales, managing **service discovery**, **load balancing**, **inter-service communication**, and **fault tolerance** becomes increasingly complex.
- Failures or latency in one microservice can propagate across the system, requiring loosely coupled services and resilient communication patterns to maintain overall reliability.

2.3 Integrating MLOps with DVC Pipeline

- The integration of machine learning for dynamic ambulance pricing introduces additional complexity beyond traditional microservices architectures.
- Implementing an end-to-end **MLOps pipeline with DVC** requires strict versioning of datasets, preprocessing steps, and trained models to ensure reproducibility and traceability.
- Deploying updated models into live, containerized services demands robust CI/CD workflows to validate, version, and integrate models without disrupting ongoing emergency operations.

3 Main Architecture

Figure 6 illustrates the architecture of the Smart Emergency Response System and its integration with an MLOps-driven pricing pipeline. Client users initiate emergency requests through the **Emergency Service**, which forwards the request to the **Dispatch Service** for coordination. The Dispatch Service

acts as the central orchestrator, communicating with the **Ambulance Service** to manage ambulance availability and operational details, including administrative inputs. For dynamic pricing, the Dispatch Service invokes a dedicated **Flask API**, which serves as an interface to the machine learning infrastructure. This API triggers the **DVC-based MLOps pipeline**, ensuring that versioned data and trained models are used consistently for inference. The output of this pipeline is a real-time **ambulance price prediction**, which is returned to the Dispatch Service and propagated back to the client as part of the emergency response workflow.

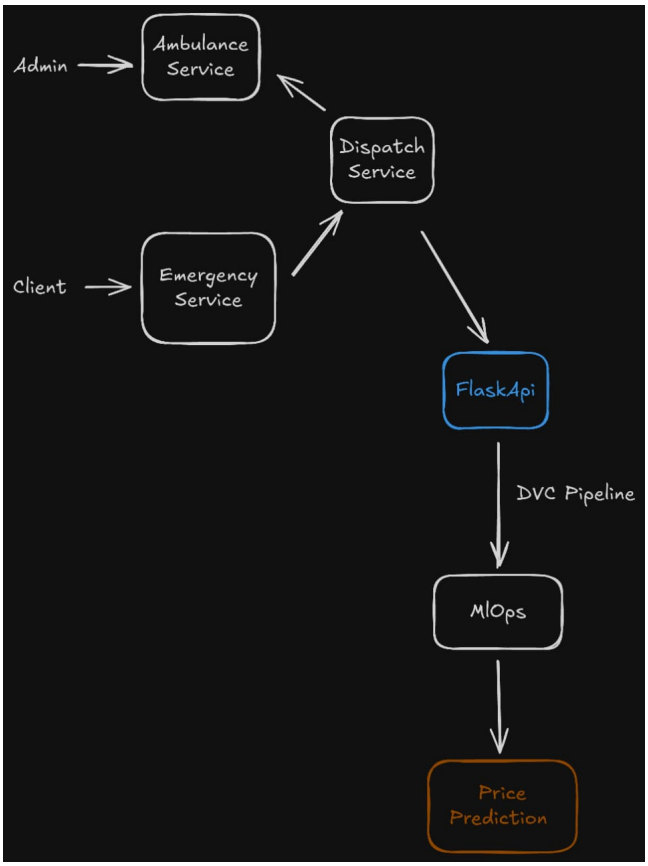

Figure 1: System Architecture of the Smart Emergency Response System

4 Results on Running Jenkins Pipeline

4.1 After Uploading first csv

The performance metrics of the model are as follows:

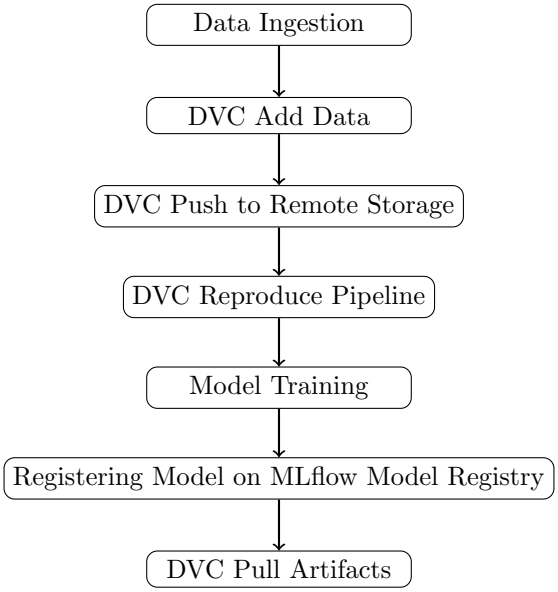

Figure 2: DVC-Based MLOps Pipeline for Model Training and Versioning

$$\text{Mean Squared Error (MSE)} = 84.65729631046247$$

$$R^2 \text{ Score} = 0.49490564355531186$$

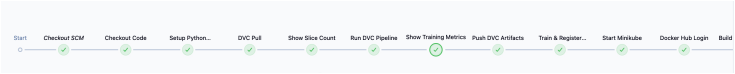

Figure 3: Jenkins Build

4.2 After Uploading second csv

Model gets trained using slice 1 and slice 2 both. The performance metrics of the model are as follows:

$$\text{Mean Squared Error (MSE)} = 23.235490579661104$$

$$R^2 \text{ Score} = 0.7635424557385845$$

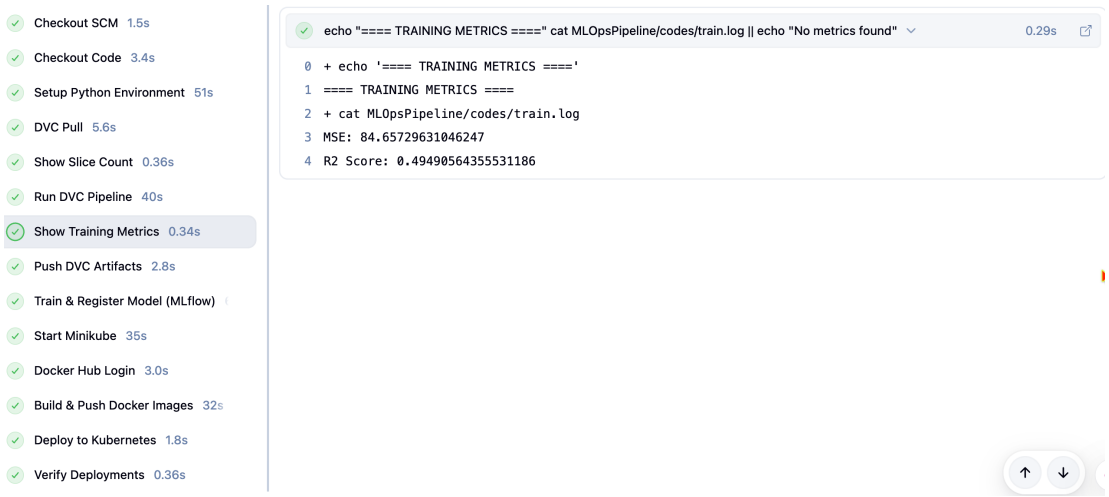

Figure 4: Output Metrics

5 Configuration Details

5.1 Dockerfile Configuration

A multi-stage Dockerfile is used to containerize the microservices in the system. The same Dockerfile is created and reused for all three services to ensure consistency, scalability, and efficient resource utilization.

5.1.1 Build Stage

The first stage uses Maven with JDK 17 to build the application.

- The base image `maven:3.9-eclipse-temurin-17` is used for compiling the Spring Boot application.
- Maven configuration files (`pom.xml`, `mvnw`, and `.mvn`) are copied first to enable Docker layer caching.
- Application dependencies are downloaded in advance to improve build performance.
- The source code is copied and the application is packaged into an executable JAR file.

5.1.2 Runtime Stage

The second stage focuses on running the application using a lightweight environment.

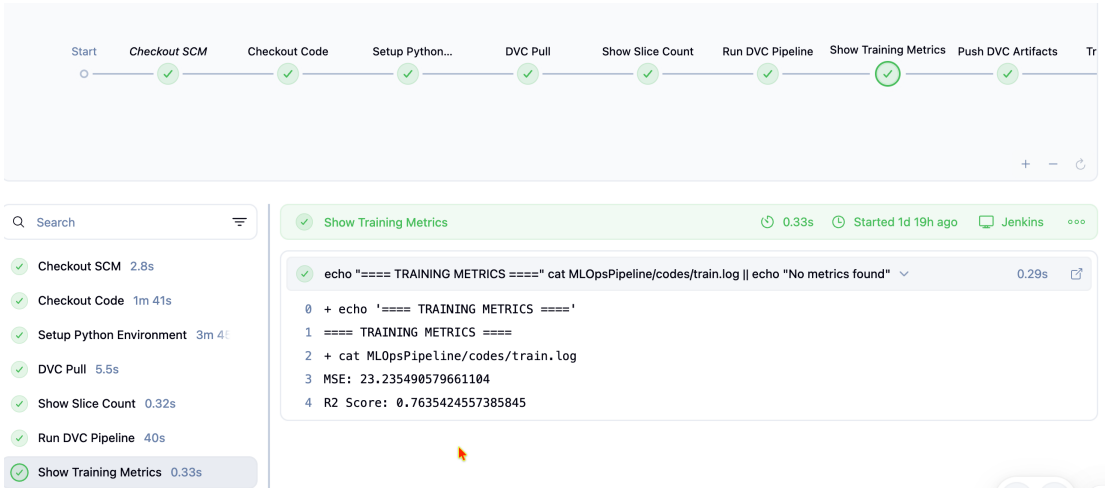

Figure 5: Output Metrics

- The base image `eclipse-temurin:17-jre` is used instead of a full JDK to reduce image size.
- Only the generated JAR file is copied from the build stage to the runtime stage.
- The application exposes port `8083` for incoming requests.
- JVM options are configured to enable container-aware memory management and optimized RAM usage.

5.1.3 Reusability Across Services

The same Dockerfile is used to containerize all three microservices by simply changing the application-specific configuration and port values. This standardized approach simplifies maintenance, ensures uniform deployment behavior, and supports scalable microservice-based architecture.

5.2 Kubernetes Configuration for Microservices

Each microservice in the system is deployed on Kubernetes using a standardized configuration consisting of a Deployment, Service, and Horizontal Pod Autoscaler (HPA). The same configuration structure is used across all services, with minor changes such as service name, container image, and port numbers.

5.2.1 Deployment Configuration

The Deployment resource is responsible for managing the lifecycle and scaling of pods.

- The application is deployed using two replicas to ensure high availability.
- Labels are used to identify and manage pods associated with the service.
- Elastic Stack logging is enabled using annotations for centralized log collection.
- CPU resource requests are defined to ensure fair scheduling and predictable performance.

5.2.2 Service Configuration

A Kubernetes Service of type `NodePort` is used to expose the application to external clients.

- The Service selects pods using application labels.
- Incoming traffic is forwarded from a static node port to the container port.
- This enables external access while maintaining internal service discovery.

5.2.3 Horizontal Pod Autoscaler

The Horizontal Pod Autoscaler (HPA) is configured to automatically scale the number of pod replicas based on CPU utilization.

- The HPA monitors average CPU utilization across all pods.
- The number of replicas is dynamically adjusted between a minimum and maximum limit.
- Pods are scaled out when CPU usage exceeds the defined threshold and scaled in when usage decreases.

5.2.4 Reusability Across Services

This Kubernetes configuration follows a uniform deployment pattern and is reused across all microservices in the system. Only service-specific fields such as application name, container image, ports, and resource limits differ, enabling simplified maintenance, consistent scalability, and easier system-wide management.

5.3 Ingress Configuration

Kubernetes Ingress is used to manage and route external HTTP traffic to multiple microservices running inside the cluster through a single entry point. This approach enables centralized access control, simplified service exposure, and efficient request routing.

5.3.1 Ingress Resource Definition

- The Ingress resource is created using the `networking.k8s.io/v1` API.
- The NGINX Ingress Controller is used, as specified by the `ingressClassName`.
- All services are exposed under a common domain name, enabling unified access to the system.

5.3.2 Path-Based Routing

The Ingress configuration uses path-based routing to forward incoming requests to appropriate backend services.

- Requests with the path prefix `/emergency` are routed to the emergency service running on port `8081`.
- Requests with the path prefix `/dispatch` are routed to the dispatch service running on port `8082`.
- Requests with the path prefix `/ambulance` are routed to the ambulance service running on port `8083`.

5.3.3 Advantages of Ingress-Based Routing

- Eliminates the need to expose each service individually using NodePort or LoadBalancer.
- Enables centralized traffic management and routing rules.
- Supports easy scalability and maintenance as services are added or updated.
- Improves security by exposing only a single external endpoint.

5.4 Centralized Logging Using ELK Stack

To enable centralized logging and real-time monitoring, the ELK stack (Elasticsearch, Filebeat, and Kibana) is deployed on the Kubernetes cluster. This setup aggregates logs generated by Kubernetes components as well as application-level logs from all microservices.

5.4.1 Elasticsearch StatefulSet

Elasticsearch is deployed as a StatefulSet to ensure stable network identity and persistent storage behavior.

- The StatefulSet runs a single-node Elasticsearch instance.
- Elasticsearch acts as the central storage and indexing engine for logs.

- Security and SSL features are disabled to simplify local cluster communication.
- Resource limits are defined to control CPU and memory usage.

A Kubernetes Service is created to expose Elasticsearch internally within the cluster, allowing other components such as Filebeat and Kibana to communicate with it.

5.4.2 Filebeat Configuration

Filebeat is used as a lightweight log shipper to collect and forward logs to Elasticsearch.

ConfigMap for Filebeat

- A ConfigMap defines Filebeat input sources and output configuration.
- Custom application log files are collected from a specified directory.
- Kubernetes autodiscovery is enabled to automatically detect pods and services.
- All collected logs are forwarded to Elasticsearch using its internal service endpoint.

RBAC Configuration

- A ServiceAccount is created for Filebeat.
- ClusterRole and ClusterRoleBinding grant Filebeat permissions to access pods, logs, namespaces, and nodes.
- This allows Filebeat to dynamically discover Kubernetes workloads and collect logs.

Filebeat DaemonSet

- Filebeat is deployed as a DaemonSet to ensure it runs on every node.
- It collects container logs, Kubernetes system logs, and manual application logs.
- Host paths are mounted to access log directories from the node filesystem.

5.4.3 Kibana Configuration

Kibana is deployed to visualize and analyze logs stored in Elasticsearch.

Kibana ConfigMap

- The ConfigMap defines Kibana server and Elasticsearch connection settings.
- Security features are disabled for simplified access.

Kibana Deployment and Service

- Kibana is deployed using a Deployment with a single replica.
- A NodePort service exposes the Kibana UI for external access.
- Users can access Kibana dashboards using the assigned node port.

5.4.4 Log Visualization in Kibana

Kibana provides a centralized interface to view, search, and analyze logs collected from the cluster.

The following types of logs can be displayed in Kibana:

- Kubernetes system logs such as pod lifecycle events and container logs.
- Logs generated by microservices deployed in the `spe-system-1` namespace.
- Custom application logs created during ambulance creation processes.
- Logs generated during emergency request creation and dispatch operations.
- Service-level logs for debugging request failures and performance monitoring.

This centralized logging approach improves observability, simplifies debugging, and enables real-time monitoring across all microservices in the system.

5.5 Jenkins CI/CD Pipeline

A declarative Jenkins pipeline is used to automate the complete MLOps and microservices deployment workflow. The same pipeline structure is followed for all services, ensuring consistency from model training to Kubernetes deployment.

5.5.1 Source Code Checkout

- The pipeline begins by cleaning the Jenkins workspace to avoid conflicts from previous builds.
- The source code is cloned from a GitHub repository using a specific branch.

5.5.2 Python Environment Setup

- A Python virtual environment is created to isolate dependencies.
- Required libraries for machine learning, DVC, Ansible, Docker, and API communication are installed.
- This ensures reproducible and consistent execution across builds.

5.5.3 Data Version Control (DVC)

- DVC is used to pull the required datasets and artifacts from a remote storage.
- The DVC pipeline is executed to reproduce model training and preprocessing stages.
- Generated artifacts and updated data versions are pushed back to the DVC remote.

5.5.4 Model Training and Registration

- Training metrics generated during model training are displayed for verification.
- The trained model is registered using MLflow for experiment tracking and version control.

5.5.5 Kubernetes Cluster Initialization

- Minikube is started to provide a local Kubernetes cluster.
- The Metrics Server addon is enabled to support autoscaling.
- Docker environment variables are configured to build images directly inside Minikube.

5.5.6 Docker Image Build and Push

- Docker images are built for all three microservices: ambulance service, dispatch service, and emergency service.
- Images are tagged and pushed to Docker Hub using secure Jenkins credentials.
- This step ensures that the latest application version is available for deployment.

5.5.7 Kubernetes Deployment

- Kubernetes manifests are applied to create namespaces, deployments, services, ingress, and autoscalers.
- All microservices are deployed automatically to the Kubernetes cluster.

5.5.8 Deployment Verification

- The status of pods in both the application namespace and default namespace is verified.
- This confirms successful deployment of microservices and monitoring components.

5.5.9 Pipeline Reusability

The Jenkins pipeline follows a standardized CI/CD workflow and is reused across all services. Only service-specific configurations such as Docker image paths and Kubernetes manifests differ. This approach enables reproducible deployments, automated scaling, and seamless integration of MLOps and DevOps practices.

5.6 Model Registry and Prediction API

To enable real-time inference using trained machine learning models, MLflow Model Registry is integrated with a REST-based prediction service built using FastAPI. This design decouples model training from model serving and enables dynamic model loading.

5.6.1 MLflow Model Registry

MLflow Model Registry is used to store, version, and manage trained machine learning models.

- The MLflow tracking server runs on a dedicated port and manages model versions and metadata.
- Trained models are registered under a unique model name in the registry.
- Each model version is stored along with its artifacts and evaluation metrics.
- The latest model version is dynamically retrieved during inference.

5.6.2 Prediction Service Using FastAPI

A FastAPI-based service is implemented to expose the prediction functionality via a REST API.

- FastAPI is chosen for its high performance and automatic request validation.
- The service accepts structured input parameters required by the machine learning model.
- Incoming requests are validated using a strongly typed schema.

5.6.3 Service Registry and Model Loading

The prediction service connects to the MLflow server to load the registered model.

- The MLflow tracking URI is configured to connect to the central MLflow server.
- The latest registered version of the model is retrieved using the MLflow client.
- The model is loaded dynamically during runtime without redeploying the service.

5.6.4 Predict Endpoint Workflow

The REST endpoint `/predict` performs real-time inference using the deployed model.

- The API receives input parameters such as pickup time, location coordinates, and passenger count.
- Input data is converted into a structured dataframe compatible with the model.
- The registered MLflow model is invoked to generate predictions.
- The predicted fare is returned in both USD and INR formats.

5.6.5 MLflow Server Configuration

The MLflow tracking server is configured as follows:

- Model metadata is stored in a SQLite backend store.
- Model artifacts such as trained weights and preprocessing steps are stored locally.
- The server is exposed on a network-accessible interface for remote inference services.

This architecture enables centralized model management, real-time inference, and seamless integration between MLOps and application services.

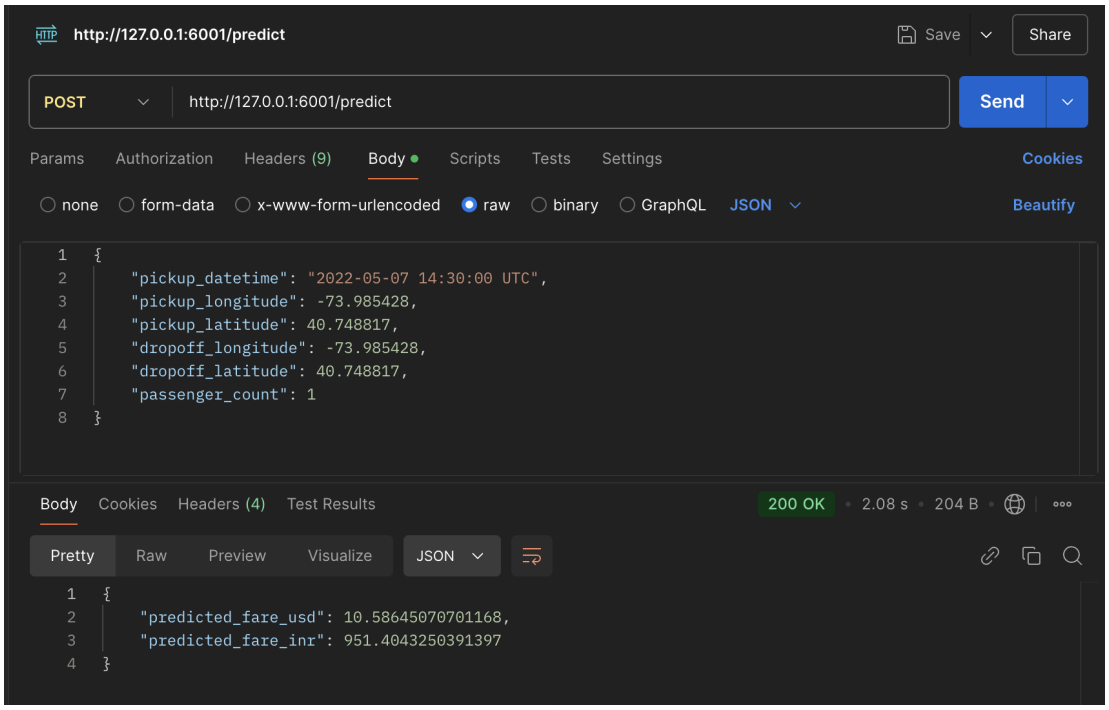

Figure 6: Postman Testing

6 Steps to run the project

6.1 Clone The Repository

```
git clone -b mlflow https://github.com/BKAPADIA04/SPE_Major_Project.git
```

6.2 Change Directory

```
cd SPE_Major_Project
```

6.3 Create Environment and Install Dependencies

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

6.4 Start MLFlow Model Registry Server

```
mlflow server \
--backend-store-uri sqlite:///mlflow.db \
```

```
--default-artifact-root ./mlruns \
--host 0.0.0.0 \
--port 5001
```

6.5 Start Flask API Server

```
export PYTHONPATH="$(pwd)/MLOpsPipeline/codes"
python3 MLflowModelAPI/mlflow_api.py
```

6.6 Start Docker Daemon and Setup Jenkins Webhook with NGROK Tunneling and Initialize DVC

```
dvc init
git commit -m "Initialize DVC"
```

6.7 Add or Remove Data from the DVC Tracked data_slices folder in MLOpsPipeline/data/data_slices

```
dvc add MLOpsPipeline/data/data_slices
dvc push
git add MLOpsPipeline/data/data_slices.dvc
git commit -m "Updated CSV data"
git push
```

6.8 Jenkins Trigger and DockerHub Image Creation and Pull

Once the dataset slice is added using `dvc add MLOpsPipeline/data/data_slices`, the corresponding DVC metadata file is generated. The data is then pushed to the configured remote storage using `dvc push`, while only the lightweight `.dvc` file is tracked in Git. After committing and pushing the changes to the repository, the Jenkins pipeline is automatically triggered via Git integration. This triggers the complete CI/CD workflow defined in the Jenkinsfile, including environment setup, data retrieval using DVC, model retraining, metric generation, model registration with MLflow, Docker image creation, and deployment of updated services to the Kubernetes cluster.